

Year Up United

Data Analyst Training Academy:

Continuing with Python

Week 6 Student Guide

This courseware is to be distributed to accompany instructor led training.

Table of Contents

Week 6 Learning Objectives	5
Week 6 Learning Outcomes.....	6
MODULE 1 JUPYTER NOTEBOOK.....	7
SECTION 1.A INTRODUCING JUPYTER NOTEBOOK	8
1.A.1 What is Jupyter Notebook?	9
1.A.2 Why Jupyter Notebook for data analysis?	10
1.A.3 Launching Jupyter Notebook.....	11
1.A.4 Creating a new notebook	12
1.A.5 Adding notebook cells	13
1.A.6 Keyboard shortcuts for Jupyter Notebook.....	14
1.A.7 Markdown for Jupyter Notebook	15
1.A.8 Viewing a shared notebook.....	16
MODULE 2 EXPLORING FUNCTIONS.....	17
SECTION 2.A BUILT-IN FUNCTIONS	18
2.A.1 Built-in functions	19
2.A.2 Finding a substring	20
2.A.3 Format function.....	21
2.A.4 Replace method	22
2.A.5 Range function	23
SECTION 2.B IMPORTING FUNCTIONS.....	24
2.B.1 Using functions from modules	25
2.B.2 Math.....	26
2.B.3 Statistics	27
2.B.4 Random.....	28
2.B.5 Datetime	29
SECTION 2.C CREATING FUNCTIONS.....	30
2.C.1 Why write your own functions?	31
2.C.2 Writing and calling functions	32
2.C.3 Parameters and arguments	33
2.C.4 Default parameters	34
2.C.5 Arbitrary arguments.....	35
2.C.6 Return statement.....	36
2.C.7 Scoping of variables	37
SECTION 2.D LAMBDA, COMPREHENSIONS & GENERATORS.....	39
2.D.1 Lambda functions.....	40
2.D.2 Why use lambdas?	41
2.D.3 Why use lambdas? (continued)	42
2.D.4 List comprehensions.....	43
2.D.5 Generator expressions.....	44
2.D.6 Generator expressions (continued)	45
2.D.7 Generator functions	46
2.D.8 Why use generator functions?	47

MODULE 3 CLASSES/OBJECTS	48
SECTION 3.A CLASSES	49
3.A.1 <i>What are classes?</i>	50
3.A.2 <i>The __init__ method</i>	51
3.A.3 <i>Adding class methods</i>	52
3.A.4 <i>Creating instances and calling methods</i>	53
SECTION 3.B CLASS ATTRIBUTES & INHERITANCE	54
3.B.1 <i>Setting default attribute values</i>	55
3.B.2 <i>Modifying attribute values</i>	56
3.B.3 <i>Applying classes to data analysis</i>	57
3.B.4 <i>Applying classes to data analysis (cont.)</i>	58
3.B.5 <i>Inheritance & child classes</i>	59
3.B.6 <i>Class inheritance in data analysis context</i>	60
3.B.7 <i>Private variables and private methods</i>	61
MODULE 4 FILES & EXCEPTIONS	62
SECTION 4.A READING & WRITING FILES.....	63
4.A.1 <i>Python's built-in file handling</i>	64
4.A.2 <i>Opening files</i>	65
4.A.3 <i>Reading files</i>	66
4.A.4 <i>Writing files</i>	67
4.A.5 <i>Closing files</i>	68
4.A.6 <i>Deleting files</i>	69
4.A.7 <i>Working with CSV files</i>	70
SECTION 4.B EXCEPTION HANDLING	71
4.B.1 <i>Handling errors</i>	72
4.B.2 <i>Syntax of try-except</i>	73
4.B.3 <i>When to use try-except</i>	74
4.B.4 <i>Try-except blocks and loops</i>	75
4.B.5 <i>Deciding which errors to report</i>	76
MODULE 5 DATA ANALYSIS LIBRARIES.....	77
SECTION 5.A OVERVIEW	78
5.A.1 <i>How is Python used for data analysis?</i>	79
5.A.2 <i>Why use Python for data analysis?</i>	80
5.A.3 <i>Overview of commonly used libraries</i>	81
5.A.4 <i>NumPy</i>	82
5.A.5 <i>pandas</i>	83
5.A.6 <i>scikit-learn</i>	84
5.A.7 <i>Matplotlib</i>	85
SECTION 5.B NUMPY BASICS.....	86
5.B.1 <i>Getting started with NumPy</i>	87
5.B.2 <i>Arrays and the ndarray object</i>	88
5.B.3 <i>Attributes of array object</i>	89
5.B.4 <i>Accessing elements of an array</i>	90
5.B.5 <i>Multidimensional arrays</i>	91

5.B.6	<i>Data types in NumPy</i>	92
5.B.7	<i>Missing values: None vs. np.nan</i>	93
5.B.8	<i>Looping over arrays</i>	94
5.B.9	<i>Week 6 Wrap-Up</i>	95

Week 6 Learning Objectives

- This week, you will learn:
 - ◇ Using Jupyter Notebook to create, organize, run, and document an analysis notebook using code and Markdown cells
 - ◇ Built-in Python functions, string methods, and standard library modules: math, statistics, random, and datetime
 - ◇ Import aliases (np, pd, plt) as standard community conventions
 - ◇ Writing custom functions with positional parameters, default values, `*args/**kwargs`, and return statements
 - ◇ Variable scope: local vs. global and its implications
 - ◇ Lambda functions and their use with `map()` and `filter()`
 - ◇ List comprehensions and generator expressions for concise data transformation and filtering
 - ◇ Defining and instantiating classes with `__init__`, instance attributes, and methods
 - ◇ Class inheritance using `super()` to extend and specialize a parent class
 - ◇ Reading from and writing to text files and CSV files using `open()` and the `csv` module
 - ◇ Handling runtime errors with `try/except/else/finally` blocks
 - ◇ The distinction between `None` (Python's absence of value) and `np.nan` (NumPy's missing numeric value)
 - ◇ Overview of NumPy, pandas, Matplotlib, and scikit-learn: what each library is used for and how to import it

Week 6 Learning Outcomes

- By the end of the week, you will be able to:
 - ◇ Write a reusable function that accepts parameters, performs a calculation, and returns a result
 - ◇ Define a class that models a real-world record type, instantiate it, and call its methods
 - ◇ Open a CSV file, read its rows, and print structured output
 - ◇ Wrap a risky operation in a try/except block and deliver a user-friendly error message
 - ◇ Explain why `np.nan` is preferred over `None` in numeric arrays

Module 1

Jupyter Notebook

Section 1.A

Introducing Jupyter Notebook

1.A.1 What is Jupyter Notebook?

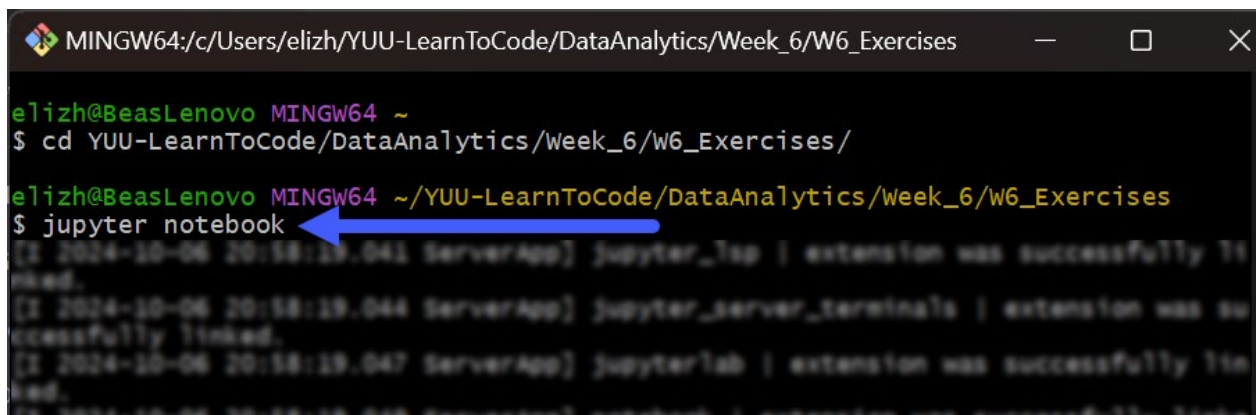
- Jupyter Notebook (and JupyterLab) can be used to create shareable and highly readable workbook-style **notebooks** that combine code blocks and text
 - ◇ Project Jupyter is an open-source project that evolved to support interactive data science and scientific computing across different programming language (starting with **Julia**, **Python**, and **R**)
- Jupyter Notebook is a web application for authoring documents that combine live-code with narrative text, equations and visualizations
 - ◇ The next generation JupyterLab is a more fully-featured interactive development environment
 - ◇ Users may install Jupyter Notebook together with Python in the Anaconda or conda package installations
- Because Jupyter Notebooks are so widely used, other applications have also developed support for viewing and editing notebook files
 - * Visual Studio Code supports working with notebook files (<https://code.visualstudio.com/docs/datascience/jupyter-notebooks>)
 - * Google Colab is another tool to view, create, edit and share notebooks (<https://colab.research.google.com/>)
 - * Binder (<https://mybinder.org/>) is a free online service for sharing a notebook repository in an interactive environment

1.A.2 Why Jupyter Notebook for data analysis?

- Jupyter Notebook is one of the preferred working environments for data analysts and data scientists because it lets you load, explore, and visualize data in the same document
 - ◇ In a notebook, you can run one cell at a time and see results *for that cell* immediately
 - ◇ This is a powerful format to break analysis down, step by step
- In Week 7, you will use Jupyter Notebook to work with the pandas, scikit-learn, and Matplotlib libraries, which are the core tools for data analysis and visualization in Python

1.A.3 Launching Jupyter Notebook

- To launch Jupyter Notebook, start the notebook server from the command line (Git Bash or Windows Command Prompt)
 - ◇ Navigate to the directory where you would like to use Jupyter Notebook
 - ◇ Launch using the command `jupyter notebook`

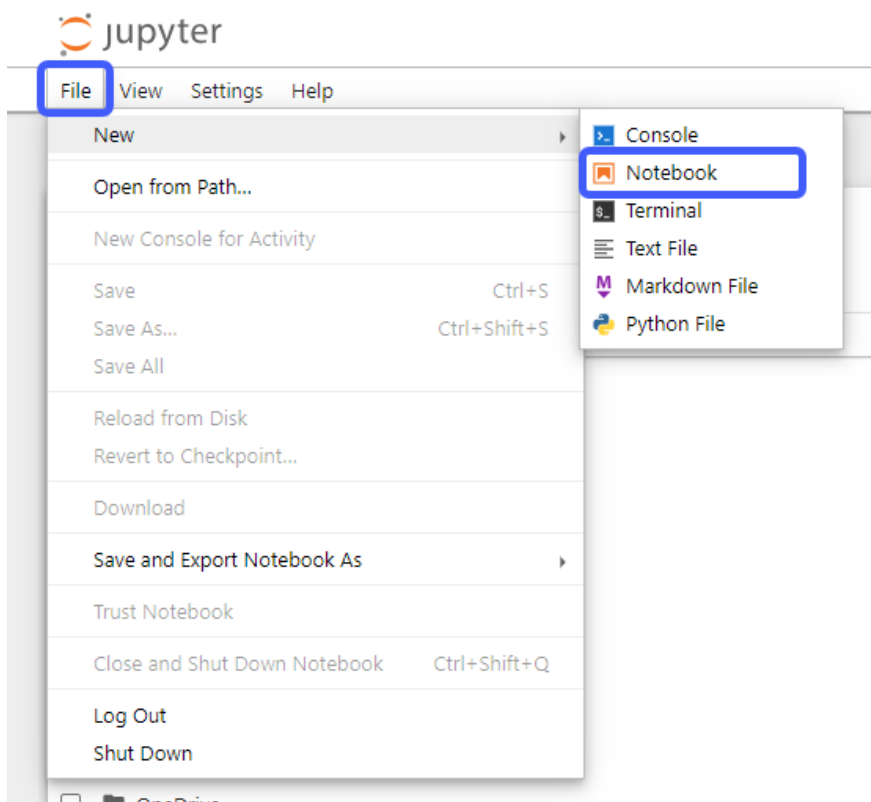


```
MINGW64:c:/Users/elizh/YUU-LearnToCode/DataAnalytics/Week_6/W6_Exercises
elizh@BeasLenovo MINGW64 ~
$ cd YUU-LearnToCode/DataAnalytics/Week_6/W6_Exercises/
elizh@BeasLenovo MINGW64 ~/YUU-LearnToCode/DataAnalytics/Week_6/W6_Exercises
$ jupyter notebook
[I 2024-10-06 20:58:19.041 ServerApp] jupyter_top | extension was successfully li
nked.
[I 2024-10-06 20:58:19.044 ServerApp] jupyter_server_terminals | extension was su
ccessfully linked.
[I 2024-10-06 20:58:19.047 ServerApp] jupyterlab | extension was successfully lin
ked.
[I 2024-10-06 20:58:19.048 ServerApp] jupyter_notebook | extension was successfully li
```

- This will start the notebook server and open Jupyter Notebook in a browser window
 - ◇ Some information about the notebook server will print to your terminal, including the URL of the web app (by default, <http://localhost:8888>)
- The browser will open to the Notebook Dashboard, with a list of the contents of the directory where the notebook server was started
 - ◇ Most of the time, you will want to start a notebook server in your home directory, where you have notebook files / project directories saved
 - ◇ More info at: <https://docs.jupyter.org/en/latest/running.html>

1.A.4 Creating a new notebook

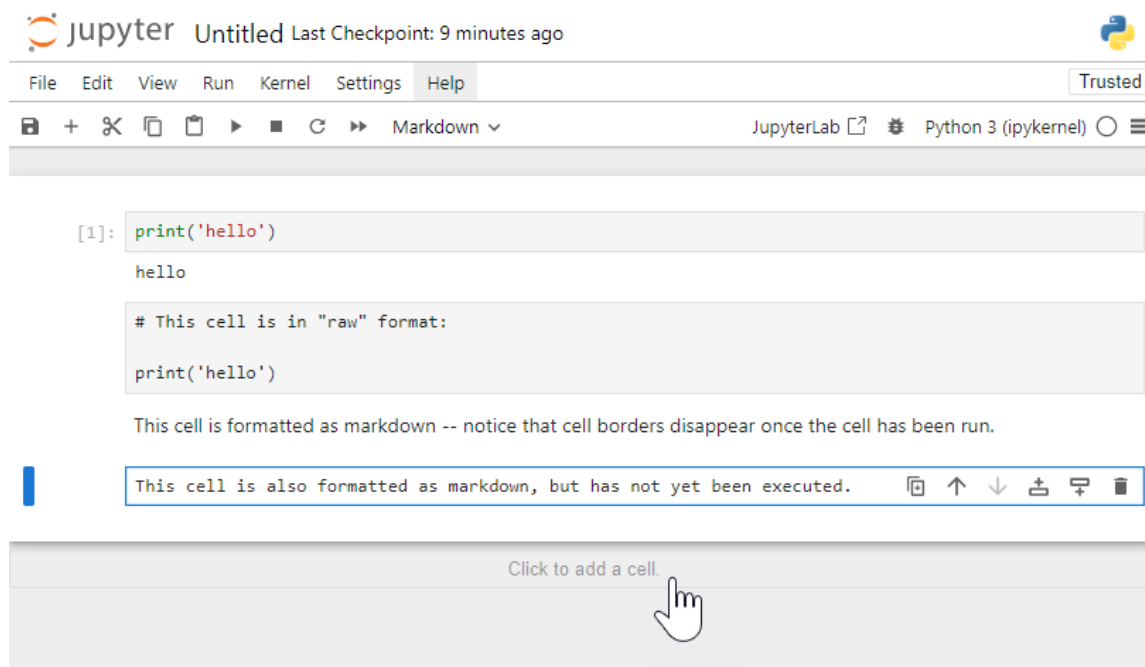
- Once you have launched Jupyter Notebook in a browser window, you can easily create a new notebook from the File menu of the dashboard
 - ◇ In addition to creating a new notebook, you can also launch a console or terminal, or create a new text file, Markdown file, or Python file



- A kernel must be selected when launching the notebook in order to run any code added to the notebook (can set default kernel)
- Once the notebook launches, click the default Untitled at the top to rename the notebook

1.A.5 Adding notebook cells

- The basic unit of a notebook document is the cell, which can be formatted as **code** (default), **markdown**, or **raw**
 - ◇ Code cells contain executable code
 - ◇ Markdown cells contain text which can be formatted using Markdown syntax, including headers, bold, italics, adding links and images, etc.
 - ◇ Raw cells contain text only – a code snippet added in raw format is not executable in the notebook



- New cells can be added in multiple ways:
 - ◇ by clicking “click to add a cell” at the bottom of the notebook
 - ◇ by clicking the + icons at the right of a cell to add a new cell above/below
 - ◇ hitting Shift+Enter (execute and advance) adds a new cell if at bottom

1.A.6 Keyboard shortcuts for Jupyter Notebook

- Keyboard shortcuts can save you a ton of time when creating notebooks, since you will use these operations over and over:
 - ◇ **Ctrl+Enter**: Run the currently selected cell
 - ◇ **Shift+Enter**: Run the currently selected cell + advance to next (if at the end of the notebook, adds a new cell below)
 - ◇ **M**: Change currently selected cell to Markdown format
 - ◇ **R**: Change currently selected cell to raw format
 - ◇ **Y**: Change currently selected cell to code format
 - ◇ **A**: Add cell above currently selected cell
 - ◇ **B**: Add cell below currently selected cell
 - ◇ **X**: Cut currently selected cell
 - ◇ **V**: Paste currently selected cell
 - ◇ **Enter**: Begin typing in currently selected cell
 - ◇ **Esc**: Stop typing in current cell without executing any code
 - ◇ **Up arrow / Down arrow**: Navigate between cells
 - ◇ **Ctrl+Shift+Q**: Close and shut down current notebook
- These and many other useful operations are also available from the **Edit menu** or by **right-clicking** on a cell

1.A.7 Markdown for Jupyter Notebook

- Markdown is a text-markup language used by Jupyter Notebook as a way to quickly and simply format text
 - ◇ A text file using Markdown can be saved with the extension `.md`
 - ◇ Many other applications, including workspaces like GitHub, also can use Markdown syntax
- Markdown syntax is designed to be readable by a human reader, even if it isn't rendered
 - ◇ For example, the formatting of:

```
For more info on the history of Markdown, see  
[Markdown] (https://en.wikipedia.org/wiki/Markdown)
```

renders as:

For more info on the history of **Markdown**, see [Markdown](#)

- Quick tips:
 - ◇ Headings are prefixed with # (H1), ## (H2) or ### (H3)
 - ◇ **bold**, *italics*, ``code``, > blockquote, ~~strikethrough~~
 - ◇ [Link name](full URL starting with https:)
 - * A useful cheat sheet: www.markdownguide.org/cheat-sheet/
 - * Also see “How to Write Faster, Better & Longer: The Ultimate Guide to Markdown,” <https://ghost.org/changelog/markdown/>

1.A.8 Viewing a shared notebook

- You can download a notebook file to your local drive, then launch Jupyter Notebook in the directory where it is saved to view it
 - ◇ Notebooks in a repository cloned to your computer will also be available to you by browsing to that local directory and launching Jupyter Notebook
- To view notebooks in a shared repository without needing to locally download anything, Binder (<https://mybinder.org/>) can be used
 - ◇ Requires link to public GitHub repository
 - ◇ Optionally may specify the specific notebook file to open
- Google Colab (<https://colab.research.google.com/>) allows you to search for notebooks by GitHub URL, organization name, or username and open, interact with, and save a copy of a notebook
 - ◇ If you run code in a notebook shared from GitHub, Colab will give you a warning message and ask you to confirm that you want to proceed
 - ◇ You must save a copy of the notebook to preserve any changes, as changes to the original file cannot be pushed back to GitHub
 - * Both of the above bullets are true even if you link your GitHub account to Colab and are viewing your own notebook

Module 2

Exploring Functions

Section 2.A

Built-in Functions

2.A.1 Built-in functions

- Python built-in functions
 - ◇ What are functions?
 - ◇ Difference between variable and function
- Examples of built-in functions
 - ◇ `print()`
 - ◇ `input()`
 - ◇ constructor functions like `str()` and `int()`
 - ◇ `range()`
 - ◇ methods like `.append()` or `.upper()`
 - ◇ functions imported from modules like `random`
 - ◇ hundreds more across the standard Python library and variable modules
- Let's explore some examples ...

2.A.2 Finding a substring

- **.find()** searches a string for a specified set of characters (could be a word or phrase), and **returns its location** using the index

◇ Example

```
order_status = "order #P080624 is ready for pickup"
print(order_status.find("is ready"))
```

output: 20 (substring starts at index 20]

```
order_status = "order #P080624 is ready for pickup"
print(order_status.find("is not ready"))
```

output: -1 (substring not found)

- **.rfind()** works like find, but searches from right to left

◇ Example

```
order_status = "order #P080624 is ready for pickup!"
print(order_status.rfind("pickup!"))
```

output: 33 (still returns starting location of the positive index)

```
order_status = "order #P080624 is ready for pickup!"
print(order_status.rfind("pickuip!"))
```

output: -1 (substring not found)

2.A.3 Format function

- The **format()** function formats the specified value(s)
 - ◇ First parameter is the value to format, second parameter is format type
 - ◇ We have already used `.format()` to create strings as a fixed decimal value and a percentage value

* Examples from Week 5 exercises:

```
bill = 25.35
tip_pct = .18
tip = bill * tip_pct
print("Tip is $" + format(tip, ".2f") +
      " at " + format(tip_pct, ".0%") )
```

- When using the **.format()** method, a placeholder value can be inserted into the string using curly brackets {}, then defined in parentheses when the format method is called

◇ Example

```
txt1 = "The ribeye is {price:.2f} dollars, yikes"
print(txt1.format(price = 49))
```

- ◇ Placeholders can be identified using named indexes like {price} or using numbered indexes like {0}, or even empty placeholders {}

```
txt2 = "I'll just have {0} and {1}".format('soup', 'salad')
txt3 = "I only have ${:.2f} to spend".format(17)
print(txt2, "\n"+txt3)
```

- w3schools has an excellent reference list for formatting types:
https://www.w3schools.com/python/ref_string_format.asp

2.A.4 Replace method

- The `.replace()` method replaces a string with another string:

◇ Example:

```
a = "Hello, World!"  
print(a.replace("H", "J").replace(", ", ""))
```

output: JelloWorld!

- w3schools has an excellent reference list for string methods:
https://www.w3schools.com/python/python_ref_string.asp

2.A.5 Range function

- The `range()` function returns a sequence of numbers

- ◇ Syntax:

```
range(start, end, step)
```

- * start is optional, defaults to 0

- * step is optional, defaults to 1

- ◇ Examples:

- `range(5)` → return numbers 0,1,2,3,4

- `range(2, 5)` → returns numbers 2,3,4

- `range(2, 5, 2)` → returns numbers 2,4

- You can use `range()` directly in a for loop

- ◇ Example:

```
for x in range(0,100,5):  
    print(x)
```

- Try it:

- ◇ Make a range iterable with the numbers 1 to a million

- * Sum them and see how fast Python can do this

- ◇ Use `range()` to count to 1000, then use this to create a new list with all these numbers squared

Section 2.B

Importing Functions

2.B.1 Using functions from modules

- There are many useful standard modules: e.g. math, statistics, random, datetime, and more
 - ◇ A module is really just a .py file defining objects and functions
- You can import a module using the **import** keyword plus the name of the module
 - ◇ A module can also be assigned an **alias** on import using **as**, for example:

```
import numpy as np
```

- * Some aliases are community-standard conventions that you will frequently encounter (e.g. numpy as np, pandas as pd, matplotlib.pyplot as plt)

- ◇ You can also import just a portion of a module using **from**, for example:

```
from datetime import datetime  
from matplotlib import pyplot as plt
```

- Some modules are installed as part of the default Python package, while others can be installed using PIP (a package manager that is typically installed along with Python)
 - ◇ If for some reason you do not have PIP installed, you can download and install it from this page: <https://pypi.org/project/pip/>
 - ◇ To see the list of packages installed on your system, use `pip list` in the command line
 - ◇ Packages available for installation can be found at <https://pypi.org/>

2.B.2 Math

- In Week 5 we talked a little bit about the **math** module which extends available mathematical functions beyond the standard library
- Methods and constants available in the math module include:
 - ◇ `math.fsum()` returns the sum of all items in an iterable (tuples, arrays, lists, etc.)
 - ◇ `math.sqrt()` returns the square root of a number
 - ◇ `math.isqrt()` rounds the square root of a number down to the nearest integer
 - ◇ `math.fabs()` returns the absolute value of a number
 - ◇ `math.fmod()` returns the remainder of x/y (like % operator)
 - ◇ `math.floor()` method rounds a number down to the nearest integer; `math.ceil()` rounds a number upwards to the nearest integer; and `math.trunc()` returns the truncated integer of a number
 - ◇ `math.pi` constant returns the value of pi (3.14159...)
- w3schools has a handy reference for math module methods and constants: https://www.w3schools.com/python/module_math.asp

2.B.3 Statistics

- The **statistics** module offers quick statistical functions beyond the core math module
 - ◇ `statistics.mean()` calculates the mean (average) of the given data
 - ◇ `statistics.median()` calculates the median (middle value) of the given data
 - ◇ `statistics.mode()` calculates the mode (most frequently occurring value) of the given numeric or nominal data
 - ◇ `statistics.stdev()` calculates the standard deviation from a sample of data
 - ◇ `statistics.variance()` calculates the variance from a sample of data
- w3schools has a quick reference for the statistics module too:
https://www.w3schools.com/python/module_statistics.asp

2.B.4 Random

- The **random** module can be used to make random numbers
 - ◇ Randomness is actually a pretty deep concept, as it's hard for computers to simulate "true" randomness
 - ◇ While the random module is useful for most casual programming purposes, it is *not* suitable for use in security applications
- Functions available from the random module include:
 - ◇ `random.randint()` returns a random integer from a given range
 - ◇ `random.random()` returns a random float number between 0 and 1
 - ◇ `random.choice()` returns a random *element* from a given sequence; `random.choices()` returns a *list* with a random selection from a given sequence
 - ◇ `random.sample()` returns a sample of a given sequence (list, set, range, etc.)
 - ◇ `random.shuffle()` takes a sequence and returns it in a random order
- w3schools has a great reference for the random module as well:
https://www.w3schools.com/python/module_random.asp

2.B.5 Datetime

- There is no built-in data type for dates in the standard Python library, importing the `datetime` module allows you to work with dates as date objects
 - ◇ A date object contains year, month, day, hour, minute, second, and microsecond – which is often more detail than we want!
 - ◇ Fortunately, the `datetime` module offers many methods to return specific information about the date object

- * Example:

```
import datetime
ex_1 = datetime.datetime.now()    # current date+time
year = ex_1.year                  # returns year
day = ex_1.strftime("%A")         # returns day of week
```

- Date and time objects can be created (`.datetime`, `.date`, `.time`)

- ◇ `datetime()` requires 3 parameters: year, month, and day

- * Parameters for time (hour, minute, second, microsecond) and timezone are optional with a default of 0/None

```
baby_bday = datetime.datetime(2024, 5, 17, 18, 24)
```

- ◇ `.strftime()` method is used to format date objects into strings
- ◇ Accepts one parameter for format code, a list of which can be found at: https://www.w3schools.com/python/python_datetime.asp
- ◇ More `datetime` reference info can be found at: <https://docs.python.org/3/library/datetime.html>

Section 2.C

Creating Functions

2.C.1 Why write your own functions?

- Functions are used to execute a reusable chunk of code
 - ◇ When you write code all day, every day, or when you solve the same types of problems again and again, you start to see repeat patterns
 - ◇ You may find yourself writing the same piece of code to do the same thing over and over
 - ◇ If you can save yourself a few steps each time that pattern comes up, over and over again, you end up saving a whole lot of time!
- Creating functions helps you to modularize your code
 - ◇ A function packages together a set of code steps with an assigned name
 - ◇ By calling the function name, you can reuse that code over and over
- Functions also help teams of developers who are working on the same project
 - ◇ For instance, one team member might write a function containing a set of code to clean up data while another is working on compiling additional data sets to go through the cleanup process
 - ◇ Each team member can reuse each other's functions as they work on different aspects of the project, or import previously created functions that can be reused in a new project

2.C.2 Writing and calling functions

- A function is a block of code that is designed to perform a specific task when called
- In its simplest form, a Python function is defined using:
 - ◇ the keyword `def`
 - ◇ the name of the function
 - ◇ a set of parentheses `()` used to specify parameters
 - ◇ Syntax

```
def function_name():  
    code to be executed
```
 - ◇ Example

```
def greeting():  
    return "Hello"
```
- You call the function by using its name, followed by parentheses
 - ◇ Example

```
# This is a call to the function  
print(greeting())
```
- There is no limit to the number of lines of code in a function – they can be very short or very long

2.C.3 Parameters and arguments

- **Parameters** are defined when the function is created, which take **arguments** as inputs
 - ◇ That is, arguments are values specified when the function is called, and the variables that “catch” these values are parameters
- The function must be called with the correct number of arguments

- ◇ The following function has exactly two parameters:

```
def greeting2(time_of_day, name):  
    return f'Good {time_of_day}, {name}!'
```

- ◇ When this function is called, arguments must be supplied for both parameters, or the result will be an error
- When calling a function, you can provide arguments in two ways
 - ◇ **Positional** arguments:
`greeting2('morning', 'Bart')`
 - ◇ **Keyword** arguments:
`greeting2(name='Bart', time_of_day='morning')`

2.C.4 Default parameters

- A function can also be defined with parameters that use a default value if no argument is provided
 - ◇ Any parameters without default values must be listed first
 - ◇ Example:

```
def greeting2(name, time_of_day='day'):  
    return f'Good {time_of_day}, {name}!'
```
- Calling this function with only a `name` argument will still work, since the function can fill in the default for the second argument
 - ◇ Example:

```
print(greeting2('Bart'))
```

output: Good day, Bart!

2.C.5 Arbitrary arguments

- If you don't know how many arguments will be passed to your function, add a `*` before the catch-all parameter in the function definition
 - ◇ This will cast the `*` argument as a tuple which can then be used by this function
 - ◇ Often coders will use the generic parameter name `*args`
 - ◇ Example:

```
def greeting_kids(*args):  
    return args  
greet = greeting_kids('Henry', 'Alice')  
for name in greet:  
    print(f'Hi, {name}!')
```
- Sometimes you may want to accept an arbitrary number of arguments, without knowing exactly what kind of information will be passed to the function
 - ◇ In these cases, you can use `**` to accept an arbitrary number of **keyword arguments**, in the form of key:value pairs; function returns a dictionary
 - ◇ Often coders will use the generic parameter name `**kwargs`
 - ◇ Example:

```
def rels(rel, name, **kwargs):  
    kwargs['relationship'] = rel  
    kwargs['name'] = name  
    return kwargs  
person = rels('cousin', 'Bea', nickname='gooseface')  
print(person)
```

2.C.6 Return statement

- In Python, a function always returns a value
- Generally it is best practice to include a `return` statement that returns the desired value, but a return statement is not required
 - ◇ That is, a function can be used to do a thing but not return a value
- If no `return` statement is provided in the function, the function will return the default value of `None`

◇ Example

```
def goodmorning(name, day_name='day') :  
    print('Good morning, ' + name + '! Have a good '  
          + day_name + '!')  
print(goodmorning('Dave', 'Tuesday'))
```

- ◇ If you run the above code, the greeting will print when the function is created...
- ◇ ...but because no return statement is included in the function, the function's "return value" when called is actually `None`, and the second print statement will generate an output of `None`, like this:

output:

```
Good morning, Dave! Have a good Tuesday!  
None
```

- ◇ What's the fix here? Instead of a second print statement, just call the function using `goodmorning('Dave', 'Tuesday')`

2.C.7 Scoping of variables

- The word **scope** is used to describe how long a variable lives and where it can be accessed
- In Python, the scope of a variable can be:
 - ◇ **Global:** the variable can be accessed anywhere in the code
 - * Typically global variables are created outside of a function, but can also be created within a function using the keyword `global`
 - ◇ **Local:** this type of variable can only be used inside the code block where it is created, e.g. within a loop or a function, then expires from memory
- Sometimes this can be a little tricky to get your head around!

◇ **Example 1:**

```
x = "awesome"
def ex_func():
    x = "fantastic"
    return x
ex_func()
print("Python is " + x)
```

output: Python is awesome (x inside function is local in scope)

◇ **Example 2:**

```
x = "awesome"
def ex_func():
    x = "fantastic"
    return x
x = ex_func()
print("Python is " + x)
```

output: Python is fantastic (global variable x is assigned the value *returned by* the function)

Section 2.D

Lambdas, Comprehensions & Generators

2.D.1 Lambda functions

- A **lambda** is a small anonymous function used with one expression

- ◇ **Anonymous** means it's not named, so cannot be reused elsewhere

- ◇ The syntax for writing a lambda function is:

```
lambda parameter, [parameter2, ...]: expression
```

- ◇ The lambda function can take multiple arguments but can only have one expression

- Examples

- ◇ Using lambda to define a variable:

```
x = lambda a, b: a * b
print(x(5, 6))
```

In a dictionary:

```
t_K = {'FtoK': lambda deg_f: 273.15 + (deg_f - 32) * 5 / 9,
      'CtoK': lambda deg_c: 273.15 + (deg_c) }
print(t_K['FtoK'](32))
```

- To create a lambda and immediately call it, put the lambda in parentheses followed by the argument(s) in parentheses

- ◇ Example:

```
(lambda a, b: a * b) (2, 3)
```

- * This is essentially the same syntax as used in the first example above when calling a variable that contains a lambda and requires arguments

2.D.2 Why use lambdas?

- Lambdas can often be helpful when used with a named function

◇ Say we want to set up a calculator to return a discount amount. One approach would be to create a function like this:

```
def discount_calc1(price, pct_off):  
    return price * (1 - pct_off)  
  
print(discount_calc1(10, .3))
```

◇ Or we could create a discount function that *returns* a lambda:

```
def discount_calc2(pct_off):  
    return lambda price: price * (1 - pct_off)  
  
pct_off20 = discount_calc2(.2)  
pct_off30 = discount_calc2(.3)  
print(f'20% off: {pct_off20(10)}, 30% off:{pct_off30(10)}')
```

- * Second example sets a separate variable for each discount, which then only requires the single argument of price
- * This generates code that is a little bit longer than the first example but more modular – a little easier to read and more versatile to use

◇ A lambda can also be passed to a function as an *argument*:

```
def discount_calc3(p, f):  
    return f(p)  
  
price = 10  
group = int(input('How many? '))  
child_dis = discount_calc3(price, lambda x: x * .75)  
group_dis = discount_calc3(price,  
    lambda x: x * (1 - .2) if group > 5 else price)
```

2.D.3 Why use lambdas? (continued)

- Two of Python's built-in functions, `map()` and `filter()`, are designed to work with a function argument, and a lambda is a concise way to supply that function

◇ Examples:

- * **`map()`** applies a function to every item and returns the results

```
prices = [10, 25, 8, 40, 15]

discounted = list(map(lambda p: p * 0.8, prices))
print(discounted)    # [8.0, 20.0, 6.4, 32.0, 12.0]
```

- * **`filter()`** keeps only items where the function returns True

```
prices = [10, 25, 8, 40, 15]

expensive = list(filter(lambda p: p > 12, prices))
print(expensive)    # [25, 40, 15]
```

- * **Note:** `map()` and `filter()` return **iterator** objects (discussed on the next page); wrapping in `list()` converts the result to a plain list

2.D.4 List comprehensions

- A list comprehension is a concise way to build a list from another collection, achieving the same results as `map()` and `filter()`, written as native Python syntax

- ◇ The syntax has three parts: an expression, a loop variable, and an optional condition

```
expression for item in iterable if condition
```

- ◇ Example:

```
prices = [10, 25, 8, 40, 15]
```

```
# Transform (produces the same result as the map() example):
```

```
discounted = [p * 0.8 for p in prices]
```

```
print(discounted)  # [8.0, 20.0, 6.4, 32.0, 12.0]
```

```
# Filter (produces the same result as the filter() example):
```

```
expensive = [p for p in prices if p > 12]
```

```
print(expensive)  # [25, 40, 15]
```

- This can be a tool for more readable code as you can combine the above transform and filter examples in a single list comprehension, whereas `map()` and `filter()` would require nested expressions:

- ◇ List comprehension:

```
big_discounts = [p * 0.8 for p in prices if p > 12]
```

```
print(big_discounts)  # [20.0, 32.0, 12.0]
```

- ◇ Combo of `map()` and `filter()`:

```
big_discounts = list(map(lambda p: p * 0.8, filter(lambda p: p > 12, prices)))
```

- ◇ Both are correct and both appear regularly in professional Python code, so it is worth being comfortable reading each

2.D.5 Generator expressions

- A **generator expression** uses similar syntax as list comprehensions but produces values one at a time rather than building a complete list in memory

◇ The difference is the use of square brackets for a list comprehension

◇ Example comparison:

- * List comprehension:

```
discounted_list = [p * 0.8 for p in prices]
```

- * Generator expression (parentheses not optional, but see note below):

```
discounted_gen = (p * 0.8 for p in prices)
```

- Generator expressions are particularly helpful as arguments to aggregation functions like `sum()`, `max()`, and `min()`

◇ These functions only need one value at a time, so there is no reason to build a full list first, when you can pass the generator expression directly:

```
total      = sum(p for p in prices)           # 98
highest    = max(p for p in prices)           # 40
big_only   = sum(p for p in prices if p > 12) # 80
```

- * **Note:** In the examples above, the generator expression is the sole argument to each function, so the function's own parentheses also serve to define the generator expression, and no extra parentheses are needed.

2.D.6 Generator expressions (continued)

- Generator expressions also work with unpacking

- ◇ If each item in a collection is a tuple, you can unpack it directly in the expression, using `_` for any value you do not need:

```
sales = [  
    ('Laptop', 'East', 1200.00),  
    ('Phone', 'West', 850.00),  
    ('Tablet', 'East', 400.00),  
]
```

```
# Sum only East region sales (unpack the tuple, ignore product name)  
total = sum(amt for _, reg, amt in sales if reg == 'East')  
print(total)  # 1600.0
```

```
# Highest sale amount across all records (ignore product and region)  
top_amount = max(amt for _, _, amt in sales)  
print(top_amount)  # 1200.0
```

- So, when to use a generator expression vs. a list comprehension?
 - ◇ Use a list comprehension when you need the result *as a list*: to be able to index into it, loop over it more than once, print it, or pass it somewhere that expects a list
 - ◇ Use a generator expression when you are *feeding the result directly into* `sum()`, `max()`, `min()`, or a similar function that consumes values one at a time
- Note that like lambdas, both generator expressions and list comprehensions are **anonymous**: they have no name, are just used in place, and can't be called again elsewhere

2.D.7 Generator functions

- A generator function can be used to define a custom iterator that can be called wherever it's needed
 - ◇ An iterator is an object which executes over a sequence of items
- Generator functions use the **yield** keyword to define the return value for each iteration
 - ◇ While a normal function can only have one `return` statement, a generator function can iterate through multiple `yield` statements, and a `yield` statement can be followed by another expression

```
def greeting():  
    yield "Hello"  
    yield "How are you?"  
    how_doing = (input("Your reply: ").lower())  
    if how_doing == "good" or how_doing == "great":  
        yield "That's good. What's new?"  
    else:  
        yield "Hey, how about this weather?"  
  
for x in greeting():  
    print(x)
```

- Local variables in a generator function are saved from one call to the next, as the function is called again with each iteration

```
def four_count():  
    x = 0  
    while x < 4:  
        yield x  
        x += 1  
  
for i in four_count():  
    print(i)
```

2.D.8 Why use generator functions?

- Generator functions are helpful when you are working with a large sequence of values and don't need all of them in memory at once
- For example, if you are reading through a file with millions of rows, generating one row at a time uses far less memory than loading everything into a list first
- When the dataset is small and you need to access items multiple times or by index, a regular list is simpler and more appropriate

Module 3

Classes/Objects

Section 3.A

Classes

3.A.1 What are classes?

- Python is an **object oriented** programming language – almost everything in Python is an **object**, with the properties and methods belonging to that type of object
- A **class** is like a blueprint used to create objects
 - ◇ When you create individual objects from the class, or **instances**, each is equipped with the behavior/attributes assigned to that class
 - * Making an object from a class is called **instantiation**
- Classes look a little different from the code we've seen so far:

◇ Example

```
class SimpleDog:                                # note use of CapCase
    '''Simple model for a dog'''                 # docstring description

    def __init__(self, name, age, he_she):
        self.name = name
        self.age = age
        self.he_she = he_she

    def sit(self):                                # this is a method
        print(f'{self.name} is now sitting')

my_dog = SimpleDog('Bobo', 12, 'He') # creating an instance

print(f'My dog\'s name is {my_dog.name}')
print(f'{my_dog.he_she} is {my_dog.age} years old')
my_dog.sit()
```

3.A.2 The `__init__` method

- A function that is included in a class is known as a **method**
- The `__init__()` method initializes fields of an instance – it runs every time an instance of the class is created
 - ◇ This is similar to a constructor in Java, but it doesn't really construct anything – it initializes fields of the class
 - ◇ By convention, **`self`** is always the first argument of `__init__`
- The first block defines **instance variables** which may be included as parameters *and/or* assigned default values
 - ◇ Example:

```
class Lumber:
    def __init__(self, width, wood='pine'):
        self.wood = wood
        self.width = width
        self.length = 96

b01 = Lumber(4, 'oak', )
print(f'{b01.wood}, {b01.width}" wide, {b01.length}" long')
```

- Note that any variable prefixed with `self.` is available to every method in the class, and accessible to any instance created from the class

3.A.3 Adding class methods

- A class can be defined with any number of additional methods
 - ◇ Class methods always use the parameter `self` within parentheses
- The example below defines two simple methods:
 - ◇ Example:

```
class SimpleDog:
    def __init__(self, name, age, he_she):
        self.name = name
        self.age = age
        self.he_she = he_she
        self.behavior = 'good'

    def sit(self):
        print(f'{self.name} is now sitting')

    def describe_behavior(self):
        print(f'{self.he_she} is a {self.behavior} dog')
```

- Methods can also be used to modify an attribute:
 - ◇ Continuing the example:

```
    def birthday(self):
        self.age += 1
        print(f'Happy birthday, {self.name}!')

    def check_in(self):
        self.behavior = input(f'Being naughty or good?')
        if self.behavior.lower() == 'good':
            print(f'Yay, {self.name}, what a good dog!')
        else:
            print(f'Uh oh, naughty {self.name}!')
```

3.A.4 Creating instances and calling methods

- Once an instance of the class is created, methods can be called using dot notation

◇ Example:

```
class SimpleDog:
    def __init__(self, name, age, he_she):
        self.name = name
        self.age = age
        self.he_she = he_she
        self.behavior = 'good'

    def sit(self):
        print(f'{self.name} is now sitting')

    def describe_behavior(self):
        print(f'{self.he_she} is a {self.behavior} dog')

    def birthday(self):
        self.age += 1
        print(f'Happy birthday, {self.name}!')

my_dog = SimpleDog('Bobo', 12, 'He')
your_dog = SimpleDog('Luna', 6, 'She')

my_dog.sit()
my_dog.describe_behavior()

your_dog.sit()
your_dog.birthday()
print(f'{your_dog.name} is now {your_dog.age}')
```

Section 3.B

Class Attributes & Inheritance

3.B.1 Setting default attribute values

- Default values can be assigned for values passed in parameters in the `__init__()` method, just as default values can be set for function parameters generally
- Attributes can also be defined without being passed in a parameter, in which case a default value must be set

◇ Examples:

```
class Lumber:
    def __init__(self, width, wood='pine'):
        self.wood = wood
        self.width = width
        self.length = 96
```

```
class SimpleDog:
    def __init__(self, name, age, he_she):
        self.name = name
        self.age = age
        self.he_she = he_she
        self.behavior = 'good'
```

3.B.2 Modifying attribute values

- An attribute can be modified directly like any other variable
 - ◇ For each instance, attributes are specific to that instance, so to modify an attribute, the name of the instance must be included:

```
class Lumber:
    def __init__(self, width, wood='pine'):
        self.wood = wood
        self.width = width
```

```
inv002 = Lumber(4)
inv002.wood = 'pine'
```

- ◇ You can also modify an attribute within a class method

```
class SimpleDog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def birthday(self):
        self.age += 1
        print(f'Happy birthday, {self.name}!')
```

- Note that methods can also use and modify global variables
 - ◇ If you want an instance of a class to reference or update a global variable, it should *not* be included in the class `__init__()` method or other method
 - ◇ Instead, global variables should be defined outside the class

3.B.3 Applying classes to data analysis

- As a data analyst, you will frequently work with data that has a repeating structure, e.g. a table with many rows
 - ◇ A class can be a natural way to model that structure: one class defines what a single record looks like, and a second class collects and analyzes a set of those records
- In the Week 6 material, open `sales_data_class_example.py` to review the full script that puts together the examples below
 - ◇ **Example:** `SalesRecord` models one transaction and is the blueprint for a single row of data

```
class SalesRecord:
    '''Represents a single sales transaction.'''
    def __init__(self, store_id, product, amount, date='2025-01-01'):
        self.store_id = store_id
        self.product = product
        self.amount = amount
        self.date = date

    def summary(self):
        print(f'Store {self.store_id} | {self.product} |
${self.amount:,.2f} | {self.date}')
```

- ◇ `__init__()` takes 4 parameters: `store_id`, `product`, `amount`, and `date`
- ◇ `date` has a default value of `'2025-01-01'` and does not need to be supplied when creating an instance
- ◇ Each parameter is assigned to a `self.` attribute so it is accessible to every method in the class and to any instance created from it
- ◇ The `summary()` method prints a single formatted line for the record using an f-string

3.B.4 Applying classes to data analysis (cont.)

- ◇ Continuing the example, `SalesReport` models a collection of records and is the blueprint for an analysis of many rows:

```
class SalesReport:
    '''Collects and analyzes a set of SalesRecord objects.'''
    def __init__(self, region):
        self.region = region
        self.records = []

    def add_record(self, record):
        self.records.append(record)

    def total_sales(self):
        return sum(r.amount for r in self.records)

    def top_sale(self):
        return max(self.records, key=lambda r: r.amount)

    def print_report(self):
        print(f'\n=== {self.region} Region Report ===')
        print(f'Transactions: {len(self.records)}')
        print(f'Total Sales:  ${self.total_sales():,.2f}')
        top = self.top_sale()
        print(f'Top Sale:      {top.product} at ${top.amount:,.2f}')
```

- ◇ `self.records = []` is initialized as an empty list and grows as records are added
- ◇ `add_record(self, record)` takes a `SalesRecord` instance as an argument and appends it to `self.records`
- ◇ `total_sales()` uses `sum()` with a generator expression to add up the amount attribute across every record in `self.records`
- ◇ `top_sale()` uses `max()` with a lambda to identify the record with the highest amount and returns a `SalesRecord` instance (not just a number)
- ◇ `print_report()` calls `total_sales()` and `top_sale()` from inside the same class

3.B.5 Inheritance & child classes

- You can write child classes based on an existing parent class
 - ◇ Also referred to as subclass (child) and superclass (parent)
 - ◇ In the child's `__init__()` method, first you'll call on the `__init__()` method from the parent class to start with the same core class attributes
 - ◇ Then you can set attributes specific to the child class
 - ◇ Instances of the child class also inherit parent class methods
 - ◇ Python supports multiple inheritance (child class can have child class)

```
class GameCharacter: # parent class
    '''Basic model for all game characters'''
    def __init__(self, type, location):
        self.type = type
        self.location = location
    def char_profile(self):
        print(f'Character type: {self.type}')
        print(f'Location: {self.location}')

class Enemy(GameCharacter): # child class
    '''Basic model for hostile characters'''
    def __init__(self, type, location, strength):
        super().__init__(type, location)
        self.strength = strength

class Friendly(GameCharacter): # child class
    '''Basic model for friendly/neutral NPCs'''
    def __init__(self, type, location, quest):
        super().__init__(type, location)
        self.quest = quest

ogre = Enemy('ogre', 'World2', 60)
villager = Friendly('villager', 'Liminal', 'N')
```

3.B.6 Class inheritance in data analysis context

- In the Week 6 material, open `sales_data_class_inheritance.py` to see how the previous data analysis example can be augmented with the use of class inheritance

◇ Example, distinguishing between in-store and online sales:

```
class StoreSalesReport(SalesReport):
    '''In-store report - adds a physical store location.'''
    def __init__(self, region, store_location):
        super().__init__(region)
        self.store_location = store_location

    def print_report(self):
        print(f'Location: {self.store_location}')
        super().print_report()    # reuse parent's print_report logic

class OnlineSalesReport(SalesReport):
    '''Online report - also tracks ship-to states.'''
    def __init__(self, region):
        super().__init__(region)
        self.ship_to_states = []

    def add_record(self, record, state):
        super().add_record(record)    # reuse parent's add_record
logic
        self.ship_to_states.append(state)

    def top_state(self):
        return max(set(self.ship_to_states),
key=self.ship_to_states.count)

    def print_report(self):
        super().print_report()
        print(f'Top Ship-to State: {self.top_state()}')
```

◇ Both of the above child classes are based on the previously-created `SalesReport` class

3.B.7 Private variables and private methods

- A private variable, or private method, cannot be seen or accessed outside the methods of the class in which it's defined
- ◇ The Python convention for designating a private variable or method is simple: precede the name with a double underscore __

Example:

```
class Enemy(GameCharacter):
    '''Basic model for hostile characters'''
    def __init__(self, type, location, strength):
        super().__init__(type, location)
        self.strength = strength
        self.__roll_attack = {0, 1, 2}

    def greet(self, name):
        print(f"You've encountered an enemy {name}")

    def __attack(self, name):
        self.__roll_attack = list(self.__roll_attack)
        self.__roll = self.__roll_attack[0]
        if self.__roll == 0:
            print(f'Enemy {name} attacks!')
            print(f'Damage = {self.strength} points')
        else:
            print(f'Enemy {name} attacks!')
            print(f'Damage = {self.strength * .75} points')
```

Module 4

Files & Exceptions

Section 4.A

Reading & Writing Files

4.A.1 Python's built-in file handling

- File handling – creating, reading, updating, and deleting files – is a key part of many coding applications
- Python's standard library contains functions dealing with each of these aspects of utilizing files in your code
- Proper sequence for file handling is always:
 - ◇ Open the file
 - ◇ Read the file
 - ◇ Close the file

4.A.2 Opening files

- Using files generally starts with the **open ()** function
 - ◇ Takes two parameters: filename and mode (optional)
- There are four different modes for opening a file:
 - ◇ "r" (Read, default value): opens a file for reading, and returns an error if the file does not exist
 - ◇ "a" (Append): opens a file for appending, or creates the file if it does not already exist
 - ◇ "w" (Write): Opens a file for writing, creates the file if it does not exist
 - ◇ "x" (Create): Creates the specified file, returns an error if the file exists
 - * Additionally can specify whether to handle as binary ("b") or text ("t", default) – e.g. "wb" to open for writing as binary
 - ◇ If the target file is saved in the same location as your code, a simple filename is all that's needed:

```
f = open("examplefile.txt", "rt")
```

- ◇ If the file is located in a different place from your code file, you will have to specify the file path

```
f3 = open("C:\Users\janedoe\ProjectX", "x")
```

4.A.3 Reading files

- `open()` returns a file object, which has **`read()`** and **`readline()`** methods for reading the content of the file
 - ◇ `.read(n)` returns the whole text by default, but you can also specify the max number of characters to return (optional parameter `n`)

* Example:

```
mobydick = open("mobydick.txt")
print(mobydick.read(200))
```

- ◇ `.readline(n)` returns just one line at a time, but can be called multiple times to print multiple lines – optional parameter `n` to specify max number of characters to return if line is longer than `n`

* `.readline()` can be especially effective with a loop

```
print(mobydick.readline())
```

- ◇ `.readlines(n)` returns a list of strings, each representing a single line – defaults to returning all lines, or can provide optional parameter `n`
 - * As with `.read()` and `.readline()`, you can specify a maximum number of characters to read for `.readlines()` – but that number will be rounded up so that an entire line is returned, not just cut off in the middle

```
print(mobydick.readlines(200))
```

- Because file objects are their own iterators in Python, all of these functions can be run inside a loop to return file contents in chunks

4.A.4 Writing files

- You can write to an existing file with `open()` using the append ("`a`") or write ("`w`") modes

- ◇ Append will add content to the end of the file

- ◇ Write will overwrite any existing content

- ◇ Example:

```
mobydick = open("mobydick.txt", "a")
mobydick.write("Epilogue: The whale's tale...")
mobydick.close()
```

- To create a new file, use the `open()` method with the create ("`x`"), append ("`a`") or write ("`w`") modes

- ◇ Create will create a file or return an error if a file of that name exists

- ◇ Append and write will create a file only if the specified file does not exist

- ◇ Example:

```
book_review = open("mobydickreview.txt", "x")
```

4.A.5 Closing files

- Best practice is always to close the file when you are done with it using the `.close()` method
 - ◇ This lets the computer know that it can free up the memory being used by the opened file
 - ◇ Example

```
mobydick = open("mobydick.txt")
print(mobydick.read(200))
mobydick.close()
```

- You can also place your file handling instructions in a **with** block to ensure the file is automatically closed
 - ◇ Behind the scenes, with the built-in method `__exit__()` which closes the file at the conclusion of the operation(s) specified in the block
 - ◇ Example

```
with open("mobydick.txt") as mobydick:
    print(mobydick.read(200))
```

- Notice the difference in scope between the two examples above
 - ◇ In the first example, the global variable `mobydick` is created and could be reused anywhere in the code
 - ◇ In the second example, the name `mobydick` is limited in scope to the **with** block

4.A.6 Deleting files

- Python has no standard library function for deleting or renaming files – to do so, you must import the `os` module
 - ◇ This module provides a portable way of using operating system dependent functionality, and it includes methods to remove a file path or directory, or to rename a file or directory
- Functions for **deletion**:
 - ◇ `os.remove()` – Remove (delete) the file
 - * If path is a directory, an `OSError` is raised; if the file does not exist, a `FileNotFoundError` is raised
 - ◇ `os.rmdir()` – Remove (delete) the directory path
 - * If the directory does not exist or is not empty, a `FileNotFoundError` or an `OSError` is raised
- Functions for **renaming**:
 - ◇ `os.rename(src, dst)` – Rename the file or directory *src* to *dst*
 - * If *dst* exists, the operation will fail with a `FileExistsError`
 - * `os.rename(old, new)` works like `rename()` but will attempt to create any intermediate directories needed to make the new pathname good
 - ◇ `os.replace(src, dst)` – Rename the file or directory *src* to *dst*
 - * If *dst* is a non-empty directory, `OSError` will be raised; if *dst* exists and is a file, it will be replaced silently if the user has permission

4.A.7 Working with CSV files

- CSV (Comma-Separated Values) is the most common file format for tabular data, which you will encounter often as a data analyst
- Python's standard library includes the `csv` module for reading and writing CSV files without installing any additional packages

◇ Example: Reading a CSV file

```
import csv
with open('sales_data.csv', 'r') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)                # each row is a list of strings
```

◇ Example: Writing to a CSV file

```
import csv
data = [
    ['Name', 'Region', 'Sales'],
    ['Alice', 'East', 4200],
    ['Bob', 'West', 3800],
]

with open('output.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(data)
```

* Note the `newline=''` argument when opening for writing, which prevents extra blank lines on Windows

- In Week 7 you will use `pd.read_csv()` from the pandas library, which builds on this same concept and adds powerful data manipulation capabilities

Section 4.B

Exception Handling

4.B.1 Handling errors

- When an error occurs, the interpreter will normally stop and generate an error message (instead of crashing your computer!)
- Errors raised by the interpreter are known as **exceptions**
 - ◇ Exceptions are special objects used to manage errors that arise while a program is running
- Anticipating exceptions and handling them in your code will make your programs more robust and provide a better user experience when the program encounters bad data or unexpected inputs
 - ◇ An exception should only ever be an exception to a rule – if you can use other programming logic to prevent the exception being raised, that is the better way to handle it!
 - ◇ In data analysis applications, you might use error handling to address a possible `ValueError` when a column is expected to contain numbers but unexpectedly includes a non-numeric entry, or a `KeyError` where an expected column is missing from a data set
- Examples of common exceptions include `ValueError`, `NameError`, `TypeError`, `SyntaxError`, and `FileNotFoundError`
 - ◇ You have probably run into some of these already!

4.B.2 Syntax of try-except

- Exceptions are handled with **try-except** blocks, which tell the program how to handle anticipated errors and can incorporate user-friendly error messages to the person using the program
 - ◇ Using `except :` without specifying exceptions will catch any exception raised
 - ◇ Multiple exceptions *to the same action* can be listed in as a tuple within one `except` statement, or multiple `except` blocks can be included
- There are four keywords that may be used in a try-except block:
 - ◇ The **try** block lets you test a block of code for errors
 - ◇ The **except** block lets you handle the error by specifying what to do and any message to the program user
 - ◇ Optional: **else** lets you execute code when there is no error (depends on the try block running successfully)
 - ◇ Optional: **finally** lets you execute code, regardless of the result of the try and except blocks (which can be especially useful to close objects and clean up resources)
- Try-except blocks may be nested using indentation (putting a try within a try)

4.B.3 When to use try-except

- For instance, a try-except block could be used to catch a bad input from a user:

◇ Example:

```
try:
    first_n = float(input('% to calculate: '))
    answer = first_n / 100
except ValueError:
    print('Invalid input! Must be a number')
else:
    print(answer)
finally:
    print('Run again to try another number')
```

- Or might be used to catch errors when reading/writing to a file:

◇ Example – nested try blocks:

```
try:
    f = open("demofile.txt") # missing parameter "a"
    try:
        f.write("P.S. One more note...")
    except:
        print("Error, cannot write to file")
    finally:
        f.close()
except:
    print("Error, cannot open file")
```

4.B.4 Try-except blocks and loops

- The previous examples catch the exception and deliver a user-friendly message instead of a traceback, but there's no opportunity to make a correction and try again
- If you want to re-prompt for an input or otherwise re-try the code, you can place the try-except block in a loop

◇ Example:

```
print('Division calculator:')

first_n = None

while first_n == None:
    try:
        first_n = float(input('First number: '))
    except ValueError:
        print('Invalid input! Must be a number')

second_n = None
answer = None

while second_n == None:
    try:
        second_n = float(input('Second number: '))
        answer = first_n / second_n
    except (ValueError, ZeroDivisionError):
        second_n = None
        print('Invalid input! Must be a number, not 0')

print(answer)
```

4.B.5 Deciding which errors to report

- Not every except must be raised to the user – exceptions can also be passed silently
 - ◇ To silently handle an error without raising a message / response, use the keyword `pass`

```
except :  
    pass
```

Module 5

Data Analysis Libraries

Section 5.A

Overview

5.A.1 How is Python used for data analysis?

- So far you have:
 - ◇ created variables to hold different values and expressions
 - ◇ learned about using functions to interact with many different sorts of objects in Python
 - ◇ practiced creating loops to run over a collection like a list or set
 - ◇ used conditional logic to determine how to handle different possible alternatives
 - ◇ used a Python script to open, read, and write to files
 - ◇ examined different types of exceptions and how to use try-except blocks
- All of these are practical skills when it comes to analyzing data!
 - ◇ Reading from data files and looping over data
 - ◇ Creating variables, functions, loops, and if/else logic to provide data calculations or assess the quality of data
 - ◇ Performing calculations on a data set to provide useful information and insights
 - ◇ Managing exceptions in reading and manipulating data
 - ◇ Write a cleaned data set to a file for use in other applications
- Libraries like NumPy, pandas, and Matplotlib all build on the Python knowledge you have been developing over the last 2 weeks

5.A.2 Why use Python for data analysis?

- As we learned in our discussion of relational databases, some datasets can be

VERY

VERY

LARGE

- Spreadsheet applications like Excel can handle thousands of lines of data, but they get slower as more data is added – when you get to millions of records, spreadsheets are not really an option
- Additionally, Python and its libraries provide options to pull together data sources of different types for analysis, like .csv or .txt files, Excel spreadsheets, other file types like .json or .xml files, even scraped web page content

5.A.3 Overview of commonly used libraries

- Commonly used Python libraries for analyzing data include:
 - ◇ NumPy – for computational applications over data arrays
 - ◇ pandas – for data analysis and data manipulation
 - ◇ Matplotlib – for creating custom data visualizations
- These are the “big 3” we’ll focus on – but there are many others that you may encounter as well!
 - ◇ Seaborn and Plotly are other popular visualization libraries
 - * Seaborn is actually built on Matplotlib
 - ◇ SQLite provides a lightweight disk-based database that doesn’t require a separate server process and supports query writing with SQL *inside* a Python script
 - ◇ SciPy is a collection of packages that deal with scientific computing
 - ◇ statsmodels is a statistical analysis library particularly useful for classical statistics and econometrics
 - ◇ scikit-learn, TensorFlow, and PyTorch are popular libraries for machine learning and artificial intelligence

5.A.4 NumPy

- NumPy is short for "Numerical Python" and is primarily used for working with **arrays**
 - ◇ The array object in NumPy is called `ndarray` (n-dimensional array)
- NumPy arrays are optimized for faster processing
 - ◇ A NumPy array is processed up to 50 times faster than a list object using the standard Python library
 - ◇ This makes arrays particularly valuable in data science applications, where speed and resource use are important considerations
- NumPy is an open source project
 - ◇ You can find the source code in the github repository <https://github.com/numpy/numpy>
 - ◇ The library is written in a combination of Python, C, and C++
- Note: In articles and learning content, you'll often see NumPy written as Numpy or numpy. When writing code, `numpy` is written in lower case, and typically the library is imported using the alias `np`, to save typing when calling its functions:
 - ◇ Example:

```
import numpy as np
```

5.A.5 pandas

- pandas is a Python library used for working with data sets, with functions for analyzing, cleaning, exploring, and manipulating data
 - ◇ The name pandas is derived from the term "**p**anel **d**ata" (an econometrics term for data sets that include observations over multiple time periods for the same individuals) and a play on the phrase "**P**ython **d**ata **a**nalysis"
- The primary objects used in pandas are **Series** and **DataFrames**
 - ◇ Series: a one-dimensional array that holds data of any type, like a single column
 - ◇ DataFrame: a 2 dimensional data structure, like a table with rows and columns
- pandas is an open source project
 - ◇ You can find the source code for pandas at <https://github.com/pandas-dev/pandas>
- Note: Officially, the name “pandas” should always be lower case, but in articles and learning content, you may also see “Pandas.” When writing code, `pandas` is usually imported under the `pd` alias.
 - ◇ Example:

```
import pandas as pd
```

5.A.6 scikit-learn

- scikit-learn is Python's most widely used library for machine learning and predictive analysis
- It provides a consistent, easy-to-use interface for building models that learn patterns from data and then make predictions on new data
- scikit-learn is built on top of NumPy and pandas, so the arrays and DataFrames you work with throughout this week are directly usable as model inputs
- The library covers a broad range of techniques including linear regression, classification, clustering, and more
 - ◇ All models in scikit-learn follow the same three-step pattern: `fit()`, `predict()`, and `score()`
- Note: In articles and learning content, you'll often see scikit-learn also written as Scikit-Learn, Scikit-learn, or sklearn. When writing code, the library is imported using the package name `sklearn` (the hyphen is dropped). Modules and functions are typically imported directly from submodules rather than importing the top-level library all at once:
 - ◇ Example:

```
from sklearn.linear_model import LinearRegression
```

5.A.7 Matplotlib

- Matplotlib is a low level graph plotting library that can be used to create visualizations from data
- Matplotlib uses a lot of specialized vocabulary
 - ◇ Your data is graphed on **figures** (e.g., windows, Jupyter widgets, etc.), each of which can contain one or more **axes** or areas plotting data; each axes includes two (sometimes three for 3D graphs) **axis** objects
 - ◇ Basically everything visible on the Figure is an **artist**, including lines, tickmarks, labels and text, etc., and when the Figure is rendered, all of the Artists are drawn to the **canvas**
- Like NumPy and pandas, Matplotlib is an open source project
 - ◇ You can look at the source code for Matplotlib at <https://github.com/matplotlib/matplotlib>
 - ◇ Matplotlib is mostly written in Python, with a few segments in C, Objective-C and Javascript
- Note: In its own documentation, the name Matplotlib is always capitalized, but in articles and learning content, it is often written in all lower case. When calling this library to generate visualizations, `matplotlib` is usually imported using the application interface `.pyplot`, under the `plt` alias.
 - ◇ Example:

```
import matplotlib.pyplot as plt
```

Section 5.B

NumPy Basics

5.B.1 Getting started with NumPy

- NumPy can be installed using PIP from the command line:

```
pip install numpy
```

- Once NumPy is installed, you can call it in a Python script using

```
import numpy
```

◇ Typically Python developers use `import numpy as np` for efficiency

- Any time you call a function from the imported library, you must use the library name (or alias) followed by the function in dot notation

```
import numpy as np
```

```
array1 = np.array([1, 2, 3, 4, 5])
```

5.B.2 Arrays and the ndarray object

- NumPy is used to work with collections of data known as **arrays**

- ◇ The name of the array object in NumPy is **ndarray**

- ◇ To create an ndarray object, the **array()** function is used

- * Any array-like object such as a list, tuple, etc. can be passed to the `array()` function

- * Examples:

```
array1 = np.array([1, 2, 3, 4])
```

```
array2 = np.array((( 'a', 'b'), ('c', 'd')))
```

- Arrays can contain multiple **dimensions**

- ◇ A one-dimensional (1D) array contains one set of values (equivalent to a list or to a column in a table)

- ◇ A two-dimensional (2D) array represents a 2D matrix or table, and consists of an array whose elements are 1D arrays

- * A **nested array** is an array that contains arrays as its elements – such as a 2D array or 3D array

- ◇ A zero-dimensional array is a scalar, a single element of an array

- Objects within an array **must be of the same type** (e.g. int, float)

- ◇ You can check the type of object contained in an array using the attribute `.dtype`

5.B.3 Attributes of array object

- An array has **attributes** which can be access with dot notation

- ◇ `.ndim` – returns the number of dimensions in the array
- ◇ `.size` – returns the total number of elements in the array
- ◇ `.shape` – returns a tuple with the size of each dimension, listed outermost to innermost
- ◇ `.T` – transposes dimensions

- Examples:

- ◇ Using a 2-dimensional array

```
arr_2d = np.array([[1,1],[2,2],[3,3]])
```

```
print(arr_2d.ndim) :2
```

```
print(arr_2d.size) :6
```

```
print(arr_2d.shape)      :(3, 2)
```

```
print(arr_2d)      :[ [1, 1],  
                      [2, 2],  
                      [3, 3]]
```

```
print(arr_2d.T)      :[ [1, 2, 3],  
                      [1, 2, 3]]
```

5.B.4 Accessing elements of an array

- Array indexing allows you to access an array element (or slice)
 - ◇ You can access an array element by referring to its index number, similar to a standard Python list, tuple, etc., or slice an array by index range
 - ◇ Also as you're familiar with from standard Python, NumPy array indexing starts with 0, meaning that the first element has index 0, the second has index 1, etc.; likewise, negative indexing (right to left) starts from -1
 - ◇ Example:

```
arr = np.array([1, 2, 3, 4])
print(arr[1:3]) : output [2, 3]
```

- Indexing also works with multi-dimensional arrays
 - ◇ Similar to standard Python, e.g., accessing elements in a list of lists
 - ◇ Example:

```
arr_2d = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('1st dim 0, 2nd dim 1:', arr_2d[0, 1]) :2
print('1st dim 1, 2nd dim 0:', arr_2d[1, 0]) :6
```

- ◇ Think of dimensions working their way outward in the brackets:

```

index:           0           1
                |-----1st dim-----|
                [1,2,3,4,5], [6,7,8,9,10]]
                |-----2nd dim-----|
index:    0 1 2 3 4      0 1 2 3 4
```

5.B.5 Multidimensional arrays

- An array can have any number of dimensions

◇ Example: 3-dimensional

```
|1-dim|                |-----2nd dim-----|
[[[1,2,3], [4,5,6]], [[7,8,9], [10,11,12]]]
|-----3rd dim-----|
```

* As assigned to an array:

```
x = np.array([
    [[1,2,3], [4,5,6]],
    [[7,8,9], [10,11,12]]
])
print(x.ndim)    :3
print(x.shape)   :(2, 2, 3)
```

◇ Example: 4-dimensional

```
|-----3rd dim-----|
|1-d|                |--2nd dim--|
[[[1,2], [3,4]], [[5,6], [7,8]], [[9,0], [1,2]], [[3,4], [5,6]]],
 [[2,1], [4,3]], [[6,5], [8,7]], [[0,1], [0,2]], [[0,3], [0,4]]]
|-----|
--4th dim-----|
```

* As assigned to an array:

```
y=np.array([
    [[1,2], [3,4]], [[5,6], [7,8]],
    [[9,0], [1,2]], [[3,4], [5,6]]
],
 [
    [[2,1], [4,3]], [[6,5], [8,7]],
    [[0,1], [0,2]], [[0,3], [0,4]]
]])
print(y.ndim)    :4
print(y.shape)   :(2, 4, 2, 2)
```

5.B.6 Data types in NumPy

- The data type of the objects contained in an array can be checked using the `.dtype` attribute ... but be aware that NumPy objects do not use the same names of data type as standard Python!
- NumPy has some extra data types and uses a different style of abbreviation (where the first character specifies the kind of data, plus a number reflecting either bytes per item or number of Unicode characters stored):
 - ◇ `i` - integer (signed)
 - ◇ `u` - unsigned integer
 - ◇ `?` - boolean
 - ◇ `f` - float
 - ◇ `c` - complex float
 - ◇ `m` - timedelta
 - ◇ `M` - datetime
 - ◇ `O` - (Python) object
 - ◇ `U` - Unicode string
 - ◇ `V` - raw data / fixed chunk of memory for other type (void scalar)
- * For detailed discussion of NumPy data types, see documentation: numpy.org/doc/stable/reference/arrays.dtypes.html (especially see “Specifying and constructing data types” and “Array-protocol type strings”)

5.B.7 Missing values: None vs. np.nan

- In Week 5 you learned that `None` is Python's way of representing the absence of a value
 - ◇ In NumPy, use of `None` in a numeric array forces the dtype of the array to object (same as an array of strings), which means mathematical operations can't be performed on that array

◇ Example:

```
arr_none = np.array([1.0, None, 3.0])
print(arr_none.dtype)      # object
print(np.sum(arr_none))    # TypeError, can't add an object array
```

- NumPy introduces **`np.nan`** (or **NaN** meaning “not a number”)
 - ◇ NaN lives natively in a float array and propagates through calculations
 - ◇ This means that for data analysis work, `np.nan` is almost always the better choice for a missing numeric value

◇ Example:

```
arr_nan = np.array([1.0, np.nan, 3.0])
print(arr_nan.dtype)      # float64
print(np.sum(arr_nan))    # NaN propagates through calculations
```

- NaN does not equal itself (this is counterintuitive, so take note!)
 - ◇ Do not use `==` to test for NaN; instead use `np.isnan()`:

```
print(np.nan == np.nan)    # False ← don't fall into the trap!
print(np.isnan(np.nan))    # True  ← correct way to test
```

5.B.8 Looping over arrays

- Loops can be used to iterate over one- or multi-dimensional arrays

◇ 1D array – loops through each element one by one

◇ N-dimensional array –

- * First level loop runs against outermost dimension; each nested loop runs against next inner dimension

- * Example:

```
arr3d=np.array([[[8,7], [6,5]], [[4,3], [2,1]]])
```

```
for x in arr3d:
    print(x)
```

output: `[[8 7]`
`[6 5]]`
`[[4 3]`
`[2 1]]`

```
for x in arr3d:
    for y in x:
        print(y)
```

output: `[8 7]`
`[6 5]`
`[4 3]`
`[2 1]`

```
for x in arr3d:
    for y in x:
        for z in y:
            print(z)
```

output: 8
7
6
5
4
3
2
1

5.B.9 Week 6 Wrap-Up

- You now have a strong foundation in core Python programming
- You've also gotten a start on one of the core libraries used in data analysis, NumPy
- Next week, you will continue to bring together everything from Weeks 5 and 6 as you work with four core data analysis libraries:
 - ◇ continuing with NumPy for numerical computing
 - ◇ pandas for loading and transforming datasets
 - ◇ scikit-learn for predictive modeling and linear regression
 - ◇ Matplotlib for creating visualizations

End of Week 6 Student Guide